Python Engineering

WEEK 02 - Data Structures | CLASS 06 - Dictionaries & Sets | EX14

Official class change: we leave class_05_lists_tuples and begin **class_06_dictionaries_sets**. We remain inside week_02_data_structures.

EX14 - Simulation configuration and results with dictionaries

A dictionary stores information as **key: value** pairs. Instead of remembering that state[0] is altitude or that the third argument is dt, we can use descriptive names.

```
config = {
    'h0': 0.0,
    'v0': 50.0,
    't0': 0.0,
    'g': 9.81,
    'dt': 0.1,
}
```

1. Dictionary: key -> value

```
config['v0']      # 50.0
config['dt']      # 0.1
config['dt'] = 0.01
config['name'] = 'Euler run'
```

Dictionaries are mutable. Keys are unique: assigning the same key again updates its value.

2. Useful starter methods

Expression	Use
d[key]	read an existing key
d.get(key)	read without failing if missing
d.keys()	view of keys
d.values()	view of values
d.items()	(key, value) pairs
key in d	check key membership
d.update({...})	update several keys

3. Set: collection of unique elements

```
required = {'h0', 'v0', 't0', 'g', 'dt'}
provided = set(config.keys())
```

A **set** removes duplicates and is designed for membership and set algebra. It is not indexed like a list.

4. Sets for configuration validation

```
missing = required - provided

if missing:
    print('Missing keys:', missing)
else:
    print('All required keys are present')
```

This is a set difference: required elements that do not appear among the provided elements.

5. Main set operations

Operation	Python	Meaning
Union	A | B	elements in A or B
Intersection	A & B	elements common to both
Difference	A - B	elements in A but not B
Symmetric difference	A ^ B	elements unique to either set
Membership	x in A	is x in the set?

6. Your mission

Create `ex14_simulation_config_dict_set.py`. You do not need to rewrite the whole simulator; the goal is to practice the structures.

1. Create a config dictionary with h0, v0, t0, g, dt.
2. Create a set with the required keys.
3. Convert config.keys() to a set.
4. Find missing keys using set difference.
5. If nothing is missing, read values from the dictionary.
6. Call your existing vertical-motion functions.
7. Store final outputs in a results dictionary.
8. Iterate over results.items() and print key/value pairs.

7. Results dictionary

```
results = {
    'max_altitude': max_h,
    'flight_time': t_f,
    'final_velocity': v_f,
    'iterations': i,
}
```

8. Deliberate test

Once it works, temporarily remove the `'dt'` key from config. Your set-based validation should identify exactly that missing key before the simulation runs.

Learning Tracker

Concept	EX14
dict and key:value pairs	New
read/update by key	New
keys(), values(), items()	New
set and unique elements	New
set operations	New
required-field validation	New
results dictionary	New